Music Source

搜索 / 详情 / 歌词 / 音源 / 歌单

State Ledger

当前歌曲 / 队列 / 事件 / 阶段总结

AI Curation

点歌 / 今日私选 / 共听上下文 / 候选验证

SHARED LISTENING / CURATED EDITION

Cove「一起听歌」搭建与整体架构指南 • 私选版

把搜歌、播放、歌单、共同状态、AI 点歌、每日私选和跨页面接力，组织成同一场真实、可恢复的共听体验。

小红书

小红书号：427689021

READING MAP

目录

01 1. 先说结论：它不是一个播放器，而是一套共同状态系统

02 2. 能力分层

03 3. 推荐的项目结构

04 4. 两种搭建模式

05 5. 配置设计

06 6. 音源适配层

07 7. 为什么需要后端代理音频

08 8. API 设计

09 9. 共同状态与数据模型

10 10. 前端共享播放器

11 11. 跨页面“同一场共听”

12 12. 移动端音量

13 13. AI 点歌

14 14. 把真实共听状态交给 AI

15 15. 「今日私选」的核心原则

16 16. 每日私选数据流

17 17. LLM 在每日私选中的正确位置

18 18. 每日私选缓存

19 19. 每日私选前端

20 20. 网络与代理

21 21. 错误设计：让用户知道到底发生了什么

22 22. 安全与隐私

23 23. 可靠性经验：最值得提前做的十件事

24 24. 推荐的分阶段实施顺序

25 25. 测试清单

26 26. 部署前检查

27 27. 一个最小端到端示例

28 28. 可以继续扩展的方向

29 29. 最后总结

30 附录 A：建议的技术栈

31 附录 B：参考事件格式

32 附录 C：分享时的说明

小红书

小红书号：427689021

> 一套把“搜歌、播放、歌单、共同状态、AI 点歌、每日私选、跨页面接力”组织成同一个体验的实现方法。
>
> 本文是根据 Cove 核心共听架构整理的公开复刻版，只覆盖独立的搜歌、播放、共同状态、AI 点歌、每日私选和跨页面播放器，并采用推荐的结构化工具方案。它不是对私有部署全部实现细节的逐行镜像；文中已经移除私人数据、账号信息、密钥、设备路径和关系记忆。

---

1. 先说结论：它不是一个播放器，而是一套共同状态系统

普通音乐播放器主要解决三件事：

1. 找到歌曲；
2. 拿到音频；
3. 播放、暂停、切歌。

Cove 的「一起听歌」在此基础上又多解决了几件事：

- AI 可以点歌，但不能只在文字里说“我给你放了”，必须真的生成可播放卡片；
- 用户和 AI 谁选的歌、当前在听什么、来自哪个歌单，都成为真实、可查询的共同状态；
- 主聊天、共读等页面共享同一场听歌体验，不能各自播放一首造成叠音；
- 已导入歌单、播放事件、喜欢或减少推荐等反馈，会形成紧凑的品味信号；
- 「今日私选」只能从已验证、可播放的候选歌曲中选择，LLM 不能凭空发明歌曲；
- 网络失败、会员限制、浏览器自动播放限制、并发写入等情况都要有可解释的退路。

因此，比较准确的定义是：

> 「一起听歌」是一个以音乐播放为表面、以持久共同状态为核心、由 AI 和用户共同驱动的媒体协作系统。

---

2. 能力分层

推荐把系统拆成五层，而不是把所有逻辑堆在一个页面里。

小红书
小红书号：427689021

层级	主要职责	典型模块
音源适配层	搜索、歌曲详情、歌词、歌单详情、播放地址、登录态	`music.py`
HTTP/API 层	对前端提供稳定接口，代理封面和音频流	`routes/music.py`
共同状态层	保存歌单、当前歌曲、播放事件、阶段总结	`data/music_library.json`
播放器交互层	浮动播放器、队列、歌词、音质、音量、跨页面接力	`static/chat.html`、`static/reading.html`
策展与 AI 层	AI 点歌、今日私选、反馈、共同语境注入	`routes/chat.py`、`music_daily_discovery.py`

整体关系如下：



这里最重要的设计决定是：

> 所有歌曲，无论来自用户搜索、AI 点歌、导入歌单还是每日私选，最后都进入同一个播放器和同一个共同状态层。

不要为「每日私选」再造播放器，也不要让 AI 点歌走另一条音频链路。否则很快会出现双重播放、状态不同步、播放历史分裂等问题。

---

3. 推荐的项目结构

下面是一个适合从零搭建的简化目录：

小红书
小红书号：427689021

```
app/
├── main.py
├── config.py
├── music.py
├── music_daily_discovery.py
├── routes/
│   ├── music.py
│   └── chat.py
├── static/
│   ├── chat.html
│   ├── reading.html
│   ├── music_playback_session.js
│   └── common.css
├── data/
│   ├── settings.json
│   ├── music_library.json
│   └── music_daily_discovery.json
└── tests/
    ├── test_music_proxy.py
    ├── test_music_library_persistence.py
    ├── test_music_daily_discovery.py
    └── test_music_playback_session.js
```

一个轻量版本只需要：

- Python；
- FastAPI；
- Uvicorn；
- HTTPX；
- Pydantic；
- 一个普通网页或 WebView；
- 浏览器原生 `<audio>`。

`pyncm` 可以作为网易云适配方式之一，但不应成为唯一依赖。实际运行中，Python 版本变化、匿名登录失效或第三方库停更都可能让它不可用，因此应该保留公开接口或自建兼容服务作为回退。

---

4. 两种搭建模式

4.1 只读 MVP

只读版本已经可以完成：

- 搜索歌曲；
- 读取歌曲详情和歌词；
- 导入公开歌单；

小红书
小红书号：427689021

- 获取可用音源；
- 后端代理播放；
- 保存本地共听状态；
- 生成本地每日私选；
- AI 点歌。

它不需要写入用户的网易云账号，也不需要保存完整 Cookie。

4.2 登录增强版

登录版增加：

- 播放更多需要登录态的歌曲；
- 读取自己创建的歌单；
- 把当前歌曲收藏到自己的网易云歌单；
- 扫码登录并保存完整 Cookie；
- 更高概率获取会员音源。

登录态属于高敏感信息，必须遵守下面的边界：

- Cookie 只保存在服务端；
- 前端永远不接触 Cookie；
- 日志不能打印 Cookie；
- API 响应不能返回 Cookie；
- 不要把 Cookie 写进 Git、聊天记录、提示词或备份说明；
- 最好提供一键退出和清除登录态；
- 只操作用户自己拥有、明确授权的账号和歌单；
- 使用前确认相关平台条款和当地法律要求。

---

5. 配置设计

建议至少提供以下配置：

```
{
  "netease_cookie": "",
  "netease_music_u": "",
  "netease_api_base_url": "",
  "music_quality": "high"
}
```

可选环境变量：

小红书
小红书号：427689021

```
AIONSHOME_MUSIC_PROXY=http://127.0.0.1:7897
AIONSHOME_NETEASE_API_BASE_URL=http://127.0.0.1:3000
AIONSHOME_DAILY_DISCOVERY_CANDIDATE_TIMEOUT=45
AIONSHOME_DAILY_DISCOVERY_RANKING_TIMEOUT=20
```

实际分享或部署时，可以换成更通用的名字，例如：

```
MUSIC_HTTP_PROXY=...
NETEASE_API_BASE_URL=...
DAILY_DISCOVERY_TIMEOUT=...
```

配置优先级建议为：

```
运行时设置文件
    ↓
环境变量
    ↓
安全默认值
```

注意不要在代码里硬编码代理、Cookie 或账号 ID。

---

6. 音源适配层

音源适配层应该向上层提供稳定、与具体库无关的函数：

小红书
小红书号：427689021

```
def search_songs(query: str, limit: int = 10) -> list[dict]:
    ...

def get_song_detail(song_id: int) -> dict | None:
    ...

def get_song_lyrics(song_id: int) -> dict:
    ...

def get_audio_url(song_id: int, quality: str = "high") -> str | None:
    ...

def resolve_playlist_id(value: str) -> int | None:
    ...

def get_playlist_detail(playlist_id: int, limit: int = 200) -> dict:
    ...

def get_user_playlists(limit: int = 100) -> dict:
    ...

def add_song_to_playlist(playlist_id: int, song_id: int) -> dict:
    ...
```

所有歌曲最好统一成下面的紧凑结构：

```
{
  "id": 123456789,
  "name": "歌曲名",
  "artist": "歌手名",
  "album": "专辑名",
  "cover": "https://example.com/cover.jpg",
  "provider": "netease",
  "duration": 236000,
  "playable": true
}
```

不要把临时 CDN 音频 URL 当作歌曲的长期标识。音频 URL 可能短时间后失效，可靠标识应该是：

```
provider + song_id
```

例如：

小红书
小红书号：427689021

```
netease:123456789
```

6.1 登录策略

推荐登录优先级：

```
完整 Cookie
    ↓
MUSIC_U
    ↓
匿名登录
    ↓
公开接口回退
```

登录会话应设置过期时间，例如两小时。获取音源失败时可以强制刷新登录态再重试一次，但不要无限重试。

6.2 音质策略

不要只请求一个音质等级。可以建立有序回退：

```
HIGH_QUALITY_LEVELS = (
    "jymaster",
    "sky",
    "jyeffect",
    "hires",
    "lossless",
    "exhigh",
    "higher",
    "standard",
)

SMOOTH_QUALITY_LEVELS = (
    "higher",
    "standard",
    "exhigh",
    "lossless",
    "hires",
)
```

“高音质”优先高规格，“流畅”优先更容易播放的规格。拿不到时逐级降级，而不是立即判定歌曲不可播放。

小红书
小红书号：427689021

7. 为什么需要后端代理音频

前端不应该直接长期持有网易云 CDN 地址。推荐的播放地址是自己的后端：

```
GET /api/music/stream/{song_id}?quality=high
```

后端每次播放时：

1. 根据 `song_id` 获取最新音频 URL；
2. 加上正确的 `Referer` 和 `User-Agent`；
3. 转发浏览器的 `Range` 请求；
4. 流式返回音频字节；
5. 根据上游 `Content-Range` 返回 `206 Partial Content`；
6. 不缓存临时音频 URL。

基本结构：

小红书
小红书号：427689021

```
import asyncio



@router.get("/api/music/stream/{song_id}")
async def music_stream(song_id: int, request: Request, quality: str = "high"):
    # 若适配器仍是同步实现，必须移出事件循环；长期建议直接提供异步适配器。
    audio_url = await asyncio.to_thread(
        get_audio_url,
        song_id,
        quality=quality,
    )
    if not audio_url:
        return JSONResponse(
            {"error": "当前无法取得可播放音源"},
            status_code=404,
        )

    range_header = request.headers.get("range")
    upstream_headers = {
        "Referer": "https://music.163.com/",
        "User-Agent": "Mozilla/5.0",
    }
    if range_header:
        upstream_headers["Range"] = range_header

    # 使用 HTTPX 异步流式读取上游，并把 Range/Content-Range 透传给浏览器。
    # 客户端断开时，要在 finally 中关闭上游 response/client。
    ...
```

不要在 `async def` 中直接调用同步 SDK、`requests` 或同步 DNS/文件下载；它们会阻塞整个事件循环。优先为音源适配层提供原生异步接口，无法立即改造时再使用 `asyncio.to_thread`。流式响应还必须处理客户端中断并关闭上游连接，避免连接池泄漏。

这样做的价值：

- 登录态不暴露给前端；
- CDN 地址过期后可以重新解析；
- WebView 不需要直接跨域访问音源；
- 可以统一处理代理、超时和错误；
- 浏览器拖动进度条时仍能通过 Range 请求定位；
- 后续切换其他音乐来源时，前端播放接口不需要改变。

7.1 封面也建议代理

桌面 WebView 有时会因防盗链或网络策略加载失败，可以增加：

小红书
小红书号：427689021

```
GET /api/music/cover?url=...
```

但必须限制允许的主机名，例如只允许可信图片 CDN，防止把这个接口变成任意 URL 的 SSRF 代理。

---

8. API 设计

下面是一组足以支撑核心体验的 API。它是推荐契约，不表示每个部署都已经启用了全部可选端点。

8.1 搜索、详情、歌词和播放

```
GET  /api/music/search?q={关键词}&limit=10
GET  /api/music/detail/{song_id}?quality=high
GET  /api/music/lyrics/{song_id}
POST /api/music/play
GET  /api/music/stream/{song_id}?quality=high
GET  /api/music/cover?url={封面URL}
POST /api/music/netease-login/qr/start
POST /api/music/netease-login/qr/check
POST /api/music/netease-login/logout
```

二维码开始接口返回一次性登录键和二维码内容；轮询接口必须携带该键，并明确返回“等待扫码 / 等待确认 / 成功 / 过期”；退出接口要撤销服务端登录态。一次性键、Cookie 和账号标识都不得写入聊天消息或前端持久缓存。

`POST /api/music/play` 可以接受：

```
{
  "keyword": "歌曲名 歌手名"
}
```

搜索到唯一、可播放的结果时返回：

小红书
小红书号：427689021

```
{
  "ok": true,
  "status": "ready",
  "song": {
    "provider": "netease",
    "id": 123456789,
    "name": "歌曲名",
    "artist": "歌手",
    "album": "专辑",
    "cover": "/api/music/cover?url=...",
    "stream_url": "/api/music/stream/123456789?quality=high",
    "playable": true
  }
}
```

如果存在多个同名版本，不要默认拿第一首，而应返回 `status: "needs_choice"` 和候选列表；没有结果、音源不可播放或上游失败时返回 `status: "error"` 与稳定错误码。第 13 章给出了 AI 工具层在此基础上的完整结果契约。

8.2 歌单

```
POST /api/music/playlists/import
GET  /api/music/library/playlists
GET  /api/music/library/playlists/{playlist_id}
DELETE /api/music/library/playlists/{playlist_id}
GET  /api/music/netease/my-playlists
POST /api/music/netease/playlists/add-song
POST /api/music/netease/playlists/reconcile
```

导入请求：

```
{
  "url": "网易云歌单分享链接或歌单 ID",
  "limit": 200
}
```

加入用户自有歌单：

```
{
  "playlist_id": 987654321,
  "song_id": 123456789
}
```

小红书
小红书号：427689021

8.3 共听状态

```
GET  /api/music/session
POST /api/music/session/event
POST /api/music/session/summary
```

事件请求：

```
{
  "event_type": "play",
  "actor": "user",
  "song": {
    "id": 123456789,
    "name": "歌曲名",
    "artist": "歌手"
  },
  "playlist": null,
  "note": ""
}
```

推荐的 `event_type`：

```
play
pause
resume
queue_playlist
import_playlist
add_to_playlist
skip
finish
close
```

`close` 是终止性事件：用户关闭播放器、页面明确退出共听或播放器销毁时必须上报。`finish` 表示自然播完；二者都不能继续被解释为“正在共听”。

8.4 每日私选

```
GET  /api/music/daily-discovery?refresh=false&limit=20
POST /api/music/daily-discovery/feedback
```

反馈请求：

小红书
小红书号：427689021

```
{
  "song_id": 123456789,
  "action": "more_like_this"
}
```

支持的动作可以包括：

```
like
heard
more_like_this
less_like_this
dislike
```

---

9. 共同状态与数据模型

9.1 为什么不用普通播放历史

单纯保存“用户播放了某首歌”无法表达：

- 是用户选的还是 AI 选的；
- 当时是不是共同收听；
- 来自哪个歌单；
- 当前会话是播放、暂停还是恢复；
- AI 是否应该把这首歌理解为“我们正在一起听”。

因此需要一份专门的共同状态。

9.2 `music_library.json`

推荐结构：

小红书
小红书号：427689021

```
{
  "playlists": {
    "987654321": {
      "id": 987654321,
      "name": "夜晚歌单",
      "source": "netease",
      "track_count": 42,
      "updated_at": 1785000000,
      "tracks": [
        {
          "provider": "netease",
          "id": 123456789,
          "name": "歌曲名",
          "artist": "歌手",
          "album": "专辑",
          "cover": "https://..."
        }
      ]
    }
  },
  "session": {
    "current_song": {
      "provider": "netease",
      "id": 123456789,
      "name": "歌曲名",
      "artist": "歌手"
    },
    "current_playlist": {
      "id": 987654321,
      "name": "夜晚歌单"
    },
    "last_actor": "companion",
    "state": "play",
    "updated_at": 1785000100
  },
  "events": [
    {
      "event_type": "play",
      "actor": "companion",
      "song": {
        "provider": "netease",
        "id": 123456789,
        "name": "歌曲名",
        "artist": "歌手"
      },
      "playlist": null,
      "note": "",
```


小红书
小红书号:427689021

```
      "created_at": 1785000100
    }
  ],
  "summaries": [
    {
      "period": "2026-W30",
      "top_songs": [],
      "top_artists": [],
      "recommendation_clues": []
    }
  ]
}
```

数据要保持紧凑：

- 不保存音频文件；
- 不保存临时 CDN URL；
- 不保存 Cookie；
- 不保存完整歌词到共同状态；
- 不无限增长事件数组；
- 只保存前端和 AI 真正需要的字段。

例如可以只保留最近 500 条事件、最近 8 个阶段总结。

9.3 并发写入必须处理

共听状态更新非常频繁。播放、暂停、切歌、导入歌单、刷新歌单可能在相邻时间发生。

危险写法：

```
data = json.loads(path.read_text())
data["events"].append(event)
path.write_text(json.dumps(data))
```

如果两个请求同时执行，就可能发生：

1. A 读取旧文件；
2. B 导入新歌单并保存；
3. A 把旧文件加一条事件后覆盖回去；
4. 新导入的歌单消失。

正确做法是：

小红书
小红书号：427689021

```
MUSIC_LIBRARY_LOCK = threading.RLock()

def load_music_library() -> dict:
    with MUSIC_LIBRARY_LOCK:
        ...

def save_music_library(data: dict) -> None:
    payload = json.dumps(data, ensure_ascii=False, indent=2)
    with MUSIC_LIBRARY_LOCK:
        tmp = MUSIC_LIBRARY_PATH.with_suffix(".json.tmp")
        tmp.write_text(payload, encoding="utf-8")
        tmp.replace(MUSIC_LIBRARY_PATH)

def mutate_music_library(mutator):
    with MUSIC_LIBRARY_LOCK:
        library = load_music_library()
        result = mutator(library)
        save_music_library(library)
        return result
```

三个关键点：

- 整个“读—改—写”过程在同一把锁内；
- 写临时文件后原子替换；
- JSON 解析失败必须记录日志，不能静默返回空库再覆盖原文件。

这里的 `threading.RLock` **只在单个 Python 进程内有效**。如果仍用 JSON 文件作为存储，部署时必须把 Uvicorn 固定为单 worker，并确保所有写入都经过这个进程；不能开多个 worker 共写同一个 `music_library.json`。固定的 `.json.tmp` 文件名也只适用于这一前提。

一旦需要多 worker、多容器或多个后台进程，应改用以下任一方案：

- 迁移到 SQLite，并在事务中完成“读—改—写”；
- 使用数据库或可靠的跨进程锁，同时为每次写入创建唯一临时文件；
- 把写入集中到单独的状态服务，由其他进程通过 API 更新。

数据规模进一步增大后，可以把歌单、事件、会话迁移到 SQLite，但同样要保留事务边界。

9.4 远端歌单刷新不能直接覆盖本地

网易云或兼容服务偶尔只返回部分歌曲。如果把刷新结果直接覆盖本地歌单，50 首歌可能突然变成 40 首。

更安全的策略：

小红书
小红书号：427689021

```
以 provider/song_id 为主键合并
    ↓
缺少 ID 时使用 标准化歌名 + 歌手
    ↓
远端新字段更新本地已有歌曲
    ↓
远端短结果不能删除本地历史歌曲
```

只有用户明确要求“严格同步并删除远端已不存在的歌曲”时，才执行破坏性覆盖。

---

10. 前端共享播放器

10.1 一个播放会话，按入口选择正确的高层方法

播放器 2.0 不应让所有入口直接调用底层加载函数。推荐把播放会话独立到 `static/music_playback_session.js`，由它管理播放通道、持久队列、中断恢复和游标；页面只调用语义明确的高层方法：

```
// 用户直接点某首歌，或 AI 卡片临时插播。
await playMusicDirect(song.id, "user", { kind: "direct" });

// 从歌单或持久队列按游标播放。
await playMusicFromLane("playlist", 0, "companion");
await playMusicFromLane("queue", 0, "user");
```

职责边界：

- `playMusicDirect(...)`：处理直接点播与临时中断，并保存可恢复的原播放通道；
- `playMusicFromLane(...)`：处理歌单/队列的游标、上一首和下一首；
- `playMusicOnline(...)`：仅作为内部底层原语，负责详情、音源、`audio.src`、歌词、播放事件和快照，不直接暴露给卡片、歌单或跨页入口；
- `musicPlaybackSession` 模块：统一维护 `activePlaylist`、`persistentQueue`、`playbackLane`、`interruption`、`queueView` 与各通道游标。

`actor` 同样重要，它让后端和 AI 知道这首歌是谁选的；但入口来源与播放通道不能只靠 `actor` 推断。

10.2 播放器状态

前端可以维护：

小红书
小红书号:427689021

```
const playbackSessionState = musicPlaybackSession.createSession({
  currentSong: null,
  activePlaylist: null,
  persistentQueue: { tracks: [], cursor: -1 },
  playbackLane: "legacy",
  interruption: null,
  queueView: "queue",
  playMode: "sequence",
});
```

音质、音量、面板展开状态和歌词等 UI 字段可以继续由页面状态维护，不要混进播放通道状态机。

建议的播放模式：

- 顺序播放；
- 单曲重复；
- 随机播放。

内部枚举固定为 `sequence`、`repeat-one`、`shuffle`，不要再混用 `order`、`loop` 等旧名称。`sequence` 到达通道末尾时如何处理，应由通道策略决定，而不是再增加含义重叠的“列表循环”模式。

10.3 恢复快照

浏览器本地保存：

小红书
小红书号：427689021

```
{
  "version": 2,
  "saved_at": 1785000100000,
  "song": {
    "id": 123456789,
    "name": "歌曲名",
    "artist": "歌手"
  },
  "queue": [],
  "current_index": -1,
  "active_playlist": null,
  "persistent_queue": [],
  "persistent_queue_cursor": -1,
  "playback_lane": "legacy",
  "interruption": null,
  "queue_view": "queue",
  "current_time": 83.5,
  "duration": 236,
  "play_mode": "sequence",
  "quality_preference": "smooth",
  "playing": true,
  "playing_saved_at": 1785000100000
}
```

需要注意：

- 保存前确认 `<audio>` 当前绑定的 `songId` 与状态中的歌曲一致；
- 切换歌曲时不能把上一首的播放时间写给下一首；
- 普通页面重开不一定要自动展示播放器；
- 只有明确的页面切换或用户打开音乐库时才恢复可见状态；
- `playing` 只能在短时间内用于页面接力，不能让几小时前的页面状态突然自动播放。

时间单位必须写进契约：`current_time` 和快照 `duration` 使用秒；音源元数据中的歌曲 `duration` 可使用毫秒，但必须在字段说明中明确；服务端 `updated_at` / `created_at` 可统一用 Unix 秒；浏览器通过 `Date.now()` 生成的 `saved_at`、`playing_saved_at`、`sentAt` 等字段一律使用 Unix 毫秒。比较前不得把秒和毫秒直接相减。

---

11. 跨页面“同一场共听”

11.1 问题

如果主聊天和共读页面各有一个 `<audio>`：

- 主聊天正在播放 A；
- 用户到共读点击 B；
- 两个页面可能同时出声。

小红书
小红书号：427689021

11.2 轻量方案：BroadcastChannel + localStorage

使用：

```
const CHANNEL_KEY = "music_together_channel_v1";
const TAKEOVER_KEY = "music_together_takeover_v1";
const RESUME_KEY = "music_together_resume_v1";
const HANDOFF_KEY = "music_together_playing_handoff_v1";

const channel =
  "BroadcastChannel" in window
    ? new BroadcastChannel(CHANNEL_KEY)
    : null;
```

开始或恢复播放时广播：

```
function announceMusicTakeover(snapshot) {
  channel?.postMessage({
    type: "takeover",
    snapshot,
    sentAt: Date.now(),
  });

  localStorage.setItem(
    TAKEOVER_KEY,
    JSON.stringify({
      type: "takeover",
      snapshot,
      sentAt: Date.now(),
    })
  );
}
```

其他页面收到后：

1. 暂停自己的本地 `<audio>`；
2. 使用 `suppressPlaybackEvent` 避免把被动暂停记录成用户行为；
3. 同步歌曲、队列和进度；
4. 预加载相同歌曲；
5. 保留浮动控制器；
6. 不擅自自动播放，除非满足短时接力条件。

`BroadcastChannel` 负责同源活动页面实时通信，`storage` 事件作为兼容回退。

小红书
小红书号：427689021

11.3 更稳定方案：宿主页保活

移动浏览器和 WebView 通常禁止新页面在没有用户手势时自动播放。即使保存了 `playing=true`，新页面的 `audio.play()` 也可能被拒绝。

更可靠的方式是：

- 主聊天作为宿主页保持不卸载；
- 共读等子页面放在 iframe 或内部路由中；
- 真正的 `<audio>` 始终留在宿主页；
- 子页面通过受控消息或宿主页暴露的 `playMusicDirect(...)` / `playMusicFromLane(...)` 控制播放会话；
- 子页面检测到父级播放器可用时，隐藏自己的本地播放器。

这样切换页面时不会中断音频，也不依赖新页面自动播放。

这两种方案可以同时保留：

- 宿主保活解决主路径；
- BroadcastChannel 解决多个独立窗口或标签页。

---

12. 移动端音量

部分移动浏览器，尤其 iOS Safari/WebView，可能忽略：

```
audio.volume = 0.4;
```

可以用 Web Audio：

```
const context = new AudioContext();
const source = context.createMediaElementSource(audio);
const gain = context.createGain();

source.connect(gain);
gain.connect(context.destination);

audio.volume = 1;
gain.gain.value = 0.4;
```

注意：

- 同一个 `<audio>` 只能创建一次 `MediaElementSource`；
- 必须缓存音频图；
- 用户点击播放时调用 `audioContext.resume()`；
- 滑块同时监听 `input` 和 `change`；
- Web Audio 初始化失败时回退到 `audio.volume`。

小红书
小红书号：427689021

13. AI 点歌

13.1 直接使用结构化工具调用

把点歌能力注册为模型可调用的工具。下面先使用 Cove 的**供应商无关内部定义**；这里的 `input_schema` 是 Cove 规范化字段，不是说所有上游 API 都使用同一个字段名：

```
{
  "name": "music_search_and_play",
  "description": "搜索一首真实歌曲并生成一起听卡片",
  "input_schema": {
    "type": "object",
    "properties": {
      "query": {
        "type": "string"
      }
    },
    "required": ["query"]
  }
}
```

给模型的能力说明应明确以下规则：

- 用户明确要求点歌、推荐音乐或发起一起听时，调用 `music_search_and_play`；
- `query` 填入可供真实音乐服务检索的歌曲名、歌手名或自然语言条件；
- 只有工具成功返回歌曲和播放卡片后，才能说已经找到或可以播放；
- 工具失败、没有匹配结果或候选存在歧义时，如实告知用户，不得假装已经播放。

模型发起的调用示例：

```
{
  "type": "tool_call",
  "id": "call_music_01",
  "name": "music_search_and_play",
  "arguments": {
    "query": "歌曲名 歌手名"
  }
}
```

适配不同模型供应商时，在边界层转换一次：

小红书
小红书号：427689021

Cove 内部字段	Claude 工具接口	OpenAI 兼容工具接口
`input_schema`	`input_schema`	`function.parameters`
`id`	`tool_use.id`	调用项的 `id`
`name`	`tool_use.name`	`function.name`
`arguments` 对象	`tool_use.input` 对象	解析 `function.arguments` JSON 字符串后得到

进入业务执行器前必须规范化并校验为上面的内部对象；执行完成后再用同一个 `id` 回传结果。不要把某一家供应商的原始响应格式散落到音乐业务代码中。

完整流程：

AI 点歌不只说出来，必须生成真实播放卡

01 用户	发起想听歌、推荐或一起听请求。
02 模型	发出带调用 ID 的结构化点歌请求。
03 聊天后端	规范化参数，再交给音乐业务执行器。
04 音乐适配	返回真实候选，或明确的歧义与错误。
05 工具结果	用同一调用 ID 回传 ready / needs_choice / error。
06 前端	仅在成功时渲染真实播放卡，点击后开始共听。

13.2 工具调用不能静默失败

错误做法：

1. 模型声称要播放歌曲，却没有真正调用工具；
2. 工具搜索失败后，后端仍保留“已经为你播放”的表述；
3. 工具没有返回有效歌曲或播放卡片，界面却显示成功。

正确做法是：

- 调用成功：校验工具结果，附加真实音乐卡片，再生成确认回复；
- 没有结果：明确提示未找到，并邀请用户补充歌手名或更换关键词；
- 候选有歧义：展示候选或追问用户，不擅自选取；
- 服务失败：返回可见的失败说明，并记录便于排查的错误原因；
- 所有情况都以工具实际结果为准，不把“准备调用”误写成“已经播放”。

小红书
小红书号：427689021

例如：

```
刚才没有成功连上音乐服务，所以这次还没有生成播放卡片。你可以稍后再试，或者告诉我另一首想听的歌。
```

13.3 建议的工具结果契约

工具名中的“play”表示“准备好一张可播放的真实歌曲卡片”，实际音频仍由用户点击后开始，不能在工具完成时提前写入“正在共听”。成功结果应采用稳定结构，并以 `call_id` 关联原调用：

```
{
  "ok": true,
  "status": "ready",
  "call_id": "call_music_01",
  "song": {
    "provider": "netease",
    "id": 123456789,
    "name": "歌曲名",
    "artist": "歌手名",
    "album": "专辑",
    "playable": true
  },
  "attachment": {
    "type": "music",
    "provider": "netease",
    "song_id": 123456789
  }
}
```

存在多个可信候选时，不擅自选择第一首：

小红书
小红书号：427689021

```
{
  "ok": false,
  "status": "needs_choice",
  "call_id": "call_music_01",
  "candidates": [
    {
      "provider": "netease",
      "id": 123456789,
      "name": "歌曲名",
      "artist": "歌手 A",
      "playable": true
    },
    {
      "provider": "netease",
      "id": 987654321,
      "name": "歌曲名",
      "artist": "歌手 B",
      "playable": true
    }
  ]
}
```

上游失败或没有可播放结果时返回稳定错误结构：

```
{
  "ok": false,
  "status": "error",
  "call_id": "call_music_01",
  "error": {
    "code": "MUSIC_SOURCE_UNAVAILABLE",
    "message": "音乐服务暂时无法连接"
  }
}
```

聊天后端必须核对 `call_id`、`status`、`provider/id`、`playable` 以及附件的一致性。只有 `ready` 才能生成音乐卡片；`needs_choice` 要交给用户选择；`error` 要给出可见说明。模型的自然回复是在收到工具结果后生成的第二步，不能把“自然回复 + 准备调用”当成一次已经成功的点歌。

---

14. 把真实共听状态交给 AI

不要把完整 `music_library.json` 每轮都塞进提示词。应该先根据状态和时效生成紧凑、事实化的上下文。

推荐把“活跃共听租约”固定为 2 小时（可配置）：

- `state` 为 `play` 或 `resume`，且 `updated_at` 未超过租约：才可描述为“正在共听”；

小红书
小红书号:427689021

- `pause`：可以保留当前歌曲供恢复，但只能写“已暂停”，不能写“正在共听”；
- `close`、`finish`、`ended`、`stopped`：均为终止状态，不暴露当前播放；
- 即使最后事件是 `play/resume`，一旦超过租约也视为陈旧，只能作为历史记录；
- `session.current_song` 是恢复与历史字段，不等于当前活跃播放事实。

满足活跃条件时，上下文可以是：

```
[当前共听音乐上下文]
正在共听：《歌曲名》— 歌手
已导入歌单：《夜晚歌单》（42首，示例：《A》《B》《C》）
最近共听事件：用户开始听《A》；陪伴者把《B》加入共听队列
共听沉淀：阶段性反复出现《A》《C》；推荐线索：偏爱低饱和女声和慢速电子

这些都是系统真实记录的共同状态。
“正在共听”表示双方正在一起听，而不是用户独自播放。
可以据此邀请继续听或换歌，但不能编造不存在的播放和反馈。
```

暂停、关闭或超时后应改成“最近一次共听/当前已暂停”，或者完全省略当前歌曲。页面关闭、进程重启和浏览器崩溃后的测试都必须覆盖这条规则，防止几小时前甚至几天前的歌曲重新进入最近上下文。

推荐限制：

- 当前歌曲：1 首；
- 已导入歌单：最多 6 个；
- 每个歌单示例：最多 4 首；
- 最近事件：最多 5 条；
- 阶段总结：最近 1 – 2 条；
- 不提供 Cookie、音频 URL、完整歌词或完整播放历史。

这是一种典型的“状态压缩”：

```
大规模运行数据
    ↓
确定性筛选与总结
    ↓
小块事实上下文
    ↓
对话模型
```

15. 「今日私选」的核心原则

每日私选不是从用户歌单里随机抽 20 首，也不是让 LLM 自由写一张歌单。

推荐使用“两阶段策展”：

小红书
小红书号：427689021

1. 程序寻找并验证真实候选；
2. LLM 只负责从候选中排序、挑选和写理由。

这样可以同时获得：

- 真实可播放；
- 不重复；
- 有个人品味；
- 有自然推荐理由；
- LLM 不能幻觉歌曲或 ID。

---

16. 每日私选数据流

今日私选：程序找真歌，**LLM** 只做排序和理由

Signals	Verifier	Card
品味输入	候选校验	私选卡缓存
• 导入歌单 • 共同播放事件 • 反馈与阶段总结	• ID 去重 • 排除已知歌曲 • 音源可播放验证	• 白名单排序 • 短理由 • 确定性兜底

16.1 已知歌曲排除集合

建议排除：

- 已导入歌单中的歌曲；
- 已经共同播放过的歌曲；
- 以前每日私选出现过的歌曲；
- 标记为“听过了”的歌曲；
- 标记为“不喜欢/少推这种”的歌曲；
- 同歌名同歌手的其他版本。

16.2 紧凑品味画像

可以从以下信号中提取：

- 用户最近主动播放的歌曲；
- AI 最近主动选择的歌曲；
- 高频歌手；
- 歌单名称；
- 阶段总结中的推荐线索；

小红书
小红书号：427689021

- `like` 和 `more_like_this` 的正向信号；
- `dislike` 和 `less_like_this` 的轻量负向歌手信号。

示例：

```
{
  "user_picks": [
    {"id": 1, "name": "A", "artist": "Artist A"}
  ],
  "companion_picks": [
    {"id": 2, "name": "B", "artist": "Artist B"}
  ],
  "top_artists": ["Artist A", "Artist C"],
  "playlist_names": ["夜里散步", "雨天"],
  "positive_feedback_artists": ["Artist D"],
  "avoid_artists": ["Artist E"]
}
```

16.3 向外搜索，而不是只在旧库里抽

根据画像生成有限数量查询：

```
熟悉歌手
熟悉歌单名称
正向反馈歌手
风格外扩词：indie pop、dream pop、alt r&b、city pop、jazz hip hop、neo soul……
中文、日文或地区化同义词
```

查询总数必须有上限，例如 16 个；每个查询取 10 – 15 首即可。

16.4 候选验证

每个候选必须经过：

1. ID 去重；
2. 排除已知歌曲；
3. 读取详情；
4. 获取实际音频 URL；
5. 无音源则跳过；
6. 标准化歌名和歌手；
7. 处理同歌不同版本；
8. 执行歌手多样性约束。

推荐限制：

- 最多验证 180 个候选；

小红书
小红书号：427689021

- 给 LLM 最多 160 个紧凑候选；
- 最终每位歌手最多 2 首；
- 排序候选池中每位歌手最多 6 首；
- Live、Remaster、Edit、Instrumental、Mix 等同曲变体尽量折叠。

---

17. LLM 在每日私选中的正确位置

给 LLM 的候选只包含：

```
[
  {
    "id": 123456789,
    "name": "歌曲名",
    "artist": "歌手",
    "album": "专辑"
  }
]
```

不要把以下内容发给 LLM：

- 封面 URL；
- Cookie；
- 音频 URL；
- 完整播放历史；
- 原始事件日志；
- 大量不相关聊天；
- `playable` 等内部校验字段。

提示词应要求：

- 只能选择候选列表中的 ID；
- 不得发明歌曲；
- 输出固定 JSON；
- 每首写一条短理由；
- 为每首选择一个标签；
- 整体兼顾贴近与小幅探索；
- 不要集中在一两个歌手；
- 输出无效时由程序回退。

标签示例：

小红书
小红书号：427689021

```
稳准
旁支
私心
小冒险
```

返回结构：

```
{
  "note": "今天这组前面柔一点，后面会亮起来。",
  "songs": [
    {
      "id": 123456789,
      "reason": "和你最近喜欢的慢速电子有亲缘，但鼓点更轻。",
      "tag": "旁支"
    }
  ]
}
```

程序收到后必须再次验证：

- 所有 ID 都来自候选；
- 没有重复；
- 理由长度合理；
- 标签来自白名单；
- 多样性仍满足限制；
- 数量不足时从已验证候选确定性补齐。

17.1 没有 LLM 也能工作

确定性兜底可以：

- 按用户相似信号和风格查询分组；
- 轮流从各组取歌；
- 使用固定短模板生成理由；
- 根据距离和探索程度分配标签。

每日私选的第一原则是“真实可用”，不是“必须由大模型生成”。

---

18. 每日私选缓存

建议保存在：

小红书
小红书号：427689021

```
data/music_daily_discovery.json
```

结构：

```
{
  "cards": {
    "2026-07-26": {
      "date": "2026-07-26",
      "version": 1,
      "title": "今日私选",
      "subtitle": "今天向外找了一组没有在旧歌单里出现过的歌。",
      "note": "前半段偏柔，后半段更亮。",
      "generated_at": 1785000000,
      "strategy": {
        "target_count": 20,
        "excluded_known_count": 420,
        "candidate_count": 120,
        "playable_count": 83,
        "llm_ranked": true
      },
      "songs": [
        {
          "id": 123456789,
          "name": "歌曲名",
          "artist": "歌手",
          "album": "专辑",
          "cover": "https://...",
          "reason": "推荐理由",
          "tag": "旁支",
          "playable": true
        }
      ]
    }
  },
  "feedback": [
    {
      "song_id": 123456789,
      "action": "more_like_this",
      "created_at": 1785000100
    }
  ]
}
```

缓存策略：

小红书
小红书号：427689021

- 同一天默认只生成一次；
- 再次打开直接读取缓存；
- 用户主动点“换一批”才刷新；
- 刷新超时或候选为空时，优先返回最近一张非空缓存，并携带 warning；
- 缓存读写同样使用锁和原子替换；
- 反馈数组设置上限，例如最近 1000 条；
- 运行时缓存不提交 Git。

---

19. 每日私选前端

把「今日私选」作为共享播放器中的一个标签页：

```
当前队列 | 我的歌单 | 今日私选
```

不要在页面启动时自动生成。用户打开播放器并切换到「今日私选」时再懒加载：

```
async function loadDailyDiscovery(refresh = false) {
  const query = refresh ? "?refresh=true&limit=20" : "?limit=20";
  const response = await fetch(`/api/music/daily-discovery${query}`);
  const data = await response.json();
  renderDailyDiscovery(data);
}
```

播放单首：

```
async function playDailyDiscoverySong(songId) {
  await playMusicDirect(songId, "companion", {
    kind: "daily-discovery",
    preservePanelState: true,
  });
}
```

播放全部：

小红书
小红书号：427689021

```
async function playDailyDiscoveryAll(card) {
  card.songs.forEach(song => addMusicToMixedQueue(song));
  const firstId = String(card.songs[0].id);
  const index = musicTogetherState.mixedQueue.findIndex(
    song => String(song.id) === firstId,
  );
  await playMusicFromLane("queue", Math.max(0, index), "companion");
}
```

每首歌可展示：

- 序号；
- 歌名和歌手；
- 一行推荐理由；
- 小标签；
- 反馈菜单；
- 播放按钮。

反馈菜单尽量轻：

```
喜欢 / 收藏
听过了
多推这种
少推这种
```

理由详情和反馈菜单最好互斥打开，避免移动端卡片过度拥挤。

---

20. 网络与代理

音乐链路往往比普通网页更容易遇到：

- DNS 失败；
- CDN 地区限制；
- WebView 直连失败；
- 代理只对部分域名有效；
- 本地兼容服务被错误地走代理；
- 公开接口可搜索但拿不到音源。

推荐代理规则：

小红书
小红书号：427689021

```
localhost / 127.0.0.1 / ::1 / *.local
    → 永远直连

外部网易云和 CDN
    → 显式音乐代理
    → 通用代理环境变量
    → 安全默认代理
    → 直连或明确失败
```

允许用：

```
MUSIC_HTTP_PROXY=direct
```

显式关闭代理。

不要把“搜索成功”等同于“可播放成功”。每日私选和 AI 点歌的关键路径都要单独执行 `get_audio_url()` 验证。

---

21. 错误设计：让用户知道到底发生了什么

建议区分：

情况	用户可见文案方向
搜索不到	没有找到匹配歌曲，可尝试补充歌手
歌曲存在但无音源	可能受版权、地区、会员或登录态限制
已有登录态仍失败	可能是 Cookie 过期、歌曲下架或地区限制
网易云 ID 无效	歌曲 ID 无效或歌曲已下架
网络失败	音乐服务暂时无法连接
每日私选刷新超时	继续显示最近缓存，并标明暂未刷新
浏览器拒绝自动播放	保留播放器，提示用户点一下继续

不要把所有失败都归类成“VIP 限制”，也不要让 AI 用拟人化文案掩盖技术失败。

小红书
小红书号：427689021

22. 安全与隐私

22.1 SSRF

封面代理、音频流代理和“通过分享链接导入歌单”都可能形成 SSRF。即使音频 URL 来自第三方 API，也不能因为“不是用户直接填写”就跳过校验。

必须：

- 限制协议为 HTTP/HTTPS；
- 对封面 CDN、音频 CDN、歌单分享域名分别设置最小白名单；
- 每次解析 DNS 后拒绝环回、私网、链路本地、保留地址和云元数据地址；
- 防御 DNS rebinding：连接时使用已校验地址，或由可信适配器完成固定域名请求；
- 限制重定向次数，并对每一跳重新验证协议、主机和解析后的 IP；
- 导入接口只接受可信 provider 的 ID/分享 URL，不做通用 URL 抓取；
- 设置响应大小上限和超时。

不要把任意 `audio_url` 直接交给代理。音频流最好只接收 `{provider, song_id}`，由受信任适配器解析出白名单 CDN 地址。

22.2 访问控制与滥用防护

所有会改变状态的接口——包括播放事件、歌单导入/删除、收藏、登录二维码、退出登录与每日反馈——都必须验证当前 Cove 用户及其资源权限。使用 Cookie 会话时还要做 CSRF 防护；所有入口应设置请求体大小、频率限制、超时和可审计但不含敏感值的日志。

若部署明确只允许本机访问，应绑定回环地址并在文档中写清这一信任边界；只要经局域网、反向代理或公网暴露，就不能把“前端按钮不可见”当成授权。

22.3 Cookie

- 只存服务端；
- 配置文件权限最小化；
- 不返回前端；
- 不进入 LLM；
- 不写日志；
- 不进入错误追踪上下文；
- 不进入 Git；
- 提供撤销能力。

22.4 运行数据

以下内容应加入 `.gitignore`：

小红书
小红书号：427689021

```
data/settings.json
data/music_library.json
data/music_daily_discovery.json
*.tmp
```

如果要分享项目，提供空模板：

```
settings.example.json
music_library.example.json
```

22.5 最小提示词数据

AI 只看到紧凑品味画像和真实候选，不看到：

- Cookie；
- 原始访问日志；
- 设备路径；
- 音频文件；
- 完整私人聊天；
- 完整音乐库；
- 不相关的个人信息。

---

23. 可靠性经验：最值得提前做的十件事

1. **音频必须走后端稳定路由。** 不把临时 CDN URL 存进消息或音乐库。
2. **所有写入使用事务，或在单进程 JSON 模式下使用锁和原子替换。** 避免播放事件覆盖刚导入的歌单。
3. **远端短歌单与本地历史做合并。** 不直接用短响应覆盖本地。
4. **AI 点歌失败必须可见。** 工具未成功返回歌曲和播放卡片时，不能声称已经播放。
5. **每日私选先验证再交给 LLM。** LLM 不能直接创造最终歌曲。
6. **每日卡缓存。** 每次打开都重新搜索和调用模型既慢又贵。
7. **跨页面只有一个活跃播放器。** BroadcastChannel 或宿主页保活。
8. **移动端音量准备 Web Audio 回退。**
9. **自动播放失败是正常限制。** 不要用无限重试绕过浏览器策略。
10. **“当前歌曲”必须结合终止事件和活跃租约解析。** 不依赖模型猜测，也不把持久历史误当成正在播放。

---

24. 推荐的分阶段实施顺序

阶段 A：可播放 MVP

- 搜索；

小红书
小红书号：427689021

- 歌曲详情；
- 音频 URL；
- 后端代理流；
- 简单播放卡；
- 明确错误。

验收标准：

- 搜索一首歌；
- 点击播放；
- 可暂停、拖动进度；
- 音源失败时有明确提示。

阶段 B：共同状态

- `music_library.json`；
- `session`；
- `events`；
- `actor`；
- `close` 终止事件与活跃租约；
- 锁与原子写；
- 阶段总结。

验收标准：

- 刷新后能恢复历史歌曲，但只有未过期的活跃会话才显示“正在共听”；
- AI 能区分播放、暂停、关闭和陈旧状态；
- 并发导入歌单和播放事件不会丢数据。

阶段 C：歌单与设置

- 导入网易云歌单；
- 队列；
- 搜索队列；
- 音质；
- 音量；
- 保存到用户歌单；
- 扫码登录。

阶段 D：跨页面共享

- BroadcastChannel；
- localStorage 快照；
- takeover；
- 短时 playing handoff；

小红书
小红书号：427689021

- 宿主页保活或 iframe 控制。

阶段 E：每日私选

- 已知歌曲集合；
- 品味画像；
- 搜索查询；
- 可播放候选验证；
- 确定性私选；
- 每日缓存；
- 前端标签页；
- 用户反馈。

阶段 F：LLM 策展

- 候选白名单；
- JSON 输出；
- ID 校验；
- 理由和标签；
- 超时；
- 多模型回退；
- 确定性兜底。

---

25. 测试清单

25.1 音源

- 搜索返回统一歌曲结构；
- 没有 `pyncm` 时能走回退；
- 登录态过期后只重试一次；
- 高音质失败能降级；
- 无音源时不返回伪播放地址；
- 本地地址不经过外部代理；
- 外部 CDN 使用正确代理；
- Range 请求返回正确 `206` 和 `Content-Range`。

25.2 音乐库

- 新文件得到合法空结构；
- JSON 损坏时记录错误，不覆盖原文件；
- 导入歌单和会话事件并发执行后两者都存在；
- 原子替换不会留下半写文件；
- 远端短歌单不会让本地歌单缩水；

小红书
小红书号：427689021

- 添加歌曲后远端刷新失败时，本地仍能补入歌曲。
- JSON 模式在单 worker 下运行；多 worker 模式使用事务存储或跨进程协调；
- 多进程并发不会争用同一个固定临时文件。

25.3 共同状态

- `play/pause/resume/close/finish` 正确更新 session；
- 当前歌曲不会因暂停而丢失；
- 暂停状态不会显示为“正在共听”；
- `close/finish` 后不会把历史歌曲暴露为当前播放；
- `play/resume` 超过活跃租约后自动降为历史；
- 页面关闭、进程重启和浏览器崩溃恢复后不会复活陈旧播放；
- 最近事件有长度上限；
- AI 上下文只包含白名单字段；
- 没有状态时返回空上下文而不是编造。

25.4 播放器

- 直接点播/插播使用 `playMusicDirect`，歌单/队列使用 `playMusicFromLane`；
- `playMusicOnline` 只作为内部底层加载原语；
- `activePlaylist`、`persistentQueue`、`playbackLane`、`interruption`、`queueView` 能跨刷新正确恢复；
- 切歌不会继承上一首进度；
- 各播放通道的游标正确；
- `sequence`、`repeat-one`、`shuffle` 行为正确；
- 恢复快照版本为 2，浏览器时间戳使用毫秒；
- 收到 takeover 后旧页面停止发声；
- 被动暂停不产生用户暂停事件；
- 自动播放失败后 UI 仍可继续操作；
- 移动端音量有 Web Audio 回退。

25.5 AI 点歌

- 明确点歌意图能触发结构化工具调用；
- 工具参数只包含经过校验的检索条件；
- 搜索成功并返回真实歌曲后生成卡片；
- `ready` 结果包含匹配调用 ID、provider、真实歌曲和一致的附件；
- 歧义结果返回 `needs_choice`，不会自动选择第一首；
- 搜索失败返回带稳定错误码的 `error` 并显示可见说明；
- AI 正文为空但卡片成功时仍能正常渲染；
- 多个工具调用不会造成重复卡片或乱序；
- 不会把未播放歌曲记录成正在共听。

小红书
小红书号：427689021

25.6 每日私选

- 已导入和已听歌曲被排除；
- 不可播放候选被跳过；
- 重复 ID 被删除；
- 同歌不同版本被折叠；
- 每位歌手不超过设定上限；
- LLM 不能选择候选之外的 ID；
- LLM 超时后仍能生成确定性卡片；
- 同日默认命中缓存；
- `refresh=true` 才重新生成；
- 刷新失败时返回最近非空缓存；
- 用户反馈能影响后续品味或排除集合。

---

26. 部署前检查

```
[ ] Cookie、Token、账号 ID 未进入 Git
[ ] 运行数据已加入 .gitignore
[ ] 音频和封面代理有主机限制
[ ] 歌单分享 URL、重定向每一跳和解析 IP 都经过 SSRF 校验
[ ] 变更状态的 API 有身份认证、资源授权、限流；Cookie 会话有 CSRF 防护
[ ] 音频代理支持 Range
[ ] 搜索成功与可播放成功分开判断
[ ] JSON 存储只用单 worker；多进程部署已改用事务存储或跨进程协调
[ ] 所有 JSON 写入使用进程内锁、唯一临时文件或单进程原子替换
[ ] 每日私选有缓存和确定性兜底
[ ] LLM 只能选已验证候选
[ ] 前端只有一个活跃播放器
[ ] 自动播放失败有手动继续入口
[ ] AI 点歌失败不会静默
[ ] AI 点歌成功、歧义、失败三类工具契约均通过测试
[ ] `close` 和陈旧活跃租约不会复活历史歌曲
[ ] 浏览器恢复快照为 V2，毫秒时间戳没有与服务端秒时间戳混算
[ ] 日志不包含 Cookie、音频 URL 或私人路径
[ ] 关键流程有回归测试
```

---

27. 一个最小端到端示例

用户说：

小红书
小红书号：427689021

陪我听一首适合夜里散步的歌。

系统执行：

最小端到端：真实歌曲先通过验证，再进入共同状态

01	02	03
自然语言请求	结构化工具调用	真实歌曲检索
04	05	06
详情与音源验证	播放卡与音频流	共同状态可见

只有用户真正点击播放后，系统才写入“正在共听”；工具完成只代表播放卡已经准备好。

用户第二天打开「今日私选」：

第二天打开今日私选：缓存优先，缺失时才重新策展

打开今日私选	缓存存在	缓存缺失
读取当天缓存	直接返回今日卡片	画像 → 搜索 → 验证 → 排序

One Player Surface

无论命中缓存还是重新生成，点击“播放全部”都进入同一个共享播放器队列。

28. 可以继续扩展的方向

- Spotify、Apple Music、本地音乐等多 provider 适配；
- 房间式多人同步，使用服务端 WebSocket 维护主时钟；
- 更精细的“共同品味”和“个人品味”分离；
- 专辑级连续共听；
- 共同评论、时间点批注；

小红书
小红书号：427689021

- 播放结束后的轻量情绪反馈；
- 每日私选历史馆；
- “为什么推荐这首”的证据化说明；
- 相似曲图谱；
- 本地向量检索，但只存紧凑音乐元数据；
- 跨设备播放接力；
- 系统级锁屏控制和媒体通知；
- 在合法授权前提下接入官方音乐平台 SDK。

---

29. 最后总结

Cove「一起听歌」最值得复用的，不是某个具体页面或某个网易云接口，而是下面五个架构原则：

1. **一条播放链路。** 用户点歌、AI 点歌、歌单和每日私选最终都进入同一个播放器。
2. **真实共同状态。** 当前歌曲、选择者、歌单和最近事件都由系统记录，AI 只能引用真实状态。
3. **先验证，再智能。** 歌曲必须真实存在且可播放，LLM 只在白名单候选中策展。
4. **失败不伪装。** 网络、版权、登录、自动播放和工具失败都给出准确退路。
5. **从第一天保护数据。** 锁、原子写、远端短结果合并、敏感信息隔离和缓存策略不是后期优化，而是基础能力。

如果只想快速做出一个演示，可以先完成“搜索 → 后端代理播放 → 播放卡片”。
如果想做成真正有陪伴感的「一起听歌」，共同状态、跨页面单播放器、私选候选验证和 AI 诚实边界是核心。

附录 A：建议的技术栈

后端：

```
Python 3.11+
FastAPI
Uvicorn
HTTPX
Pydantic
可选：pyncm
可选：SQLite
```

前端：

小红书
小红书号：427689021

```
HTML/CSS/JavaScript
HTMLMediaElement
Web Audio API
BroadcastChannel
localStorage
可选：WebSocket
```

附录 B：参考事件格式

```
{
  "type": "music",
  "song": {
    "id": 123456789,
    "name": "歌曲名",
    "artist": "歌手",
    "album": "专辑",
    "cover": "/api/music/cover?url=...",
    "stream_url": "/api/music/stream/123456789?quality=high"
  },
  "actor": "companion",
  "created_at": 1785000100
}
```

```
{
  "type": "music_session",
  "event_type": "play",
  "actor": "companion",
  "song_id": 123456789,
  "playlist_id": null,
  "created_at": 1785000100
}
```

```
{
  "type": "daily_discovery_feedback",
  "song_id": 123456789,
  "action": "more_like_this",
  "created_at": 1785000200
}
```

小红书
小红书号：427689021

附录 C：分享时的说明

这套架构可以用于个人项目、家庭媒体助手或受控的陪伴式应用。涉及第三方音乐平台时，应优先使用官方 SDK 或获得授权的接口，遵守账号、版权、缓存和播放相关条款。本文重点是系统架构和工程边界，不代表对任何非官方接口长期可用性的保证。

小红书
小红书号：427689021